docker_class_1(44)

monolithic architecture: all the services is deployed in one server apart from it will use one db among all services in one server is called monolithic architecture.

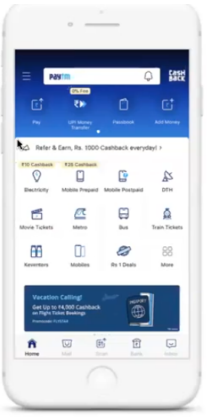
Example

Consider an application similar to **Paytm**, which may include services like:

- Movie ticket booking
- Train ticket booking
- Money transfer
- Wallet management

Defination: all these services are packaged together and deployed as **one application on a single server**.
Architecture Representation:

```
User
  |
Application Server
  |
---------------------------------
| Movie | Train | Bank | Wallet |
---------------------------------
            |
         Database
```



Disadvantages

1. **Single Point of Failure**
   If a bug occurs in one module (for example, the banking module), the entire application may need to be restarted.
2. **Difficult Scaling**
   Even if only one service requires high resources, the entire application must be scaled.
3. **Slow Deployment**
   Any small change requires redeploying the whole application.

2. Microservices Architecture

for coming micro service it is exactly opposite to monolithic
coming to here one service has one data base as include one server or every service has its own individual servers.every microservice architecture has its own database for each service.

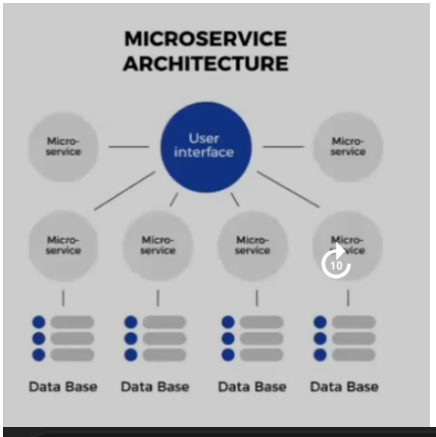
Example

Using the same **Paytm** example:

Instead of one large application, the system is split into separate services:

- Movie Ticket Service
- Train Ticket Service
- Payment Service
- Wallet Service

Each service runs independently and may have its own database.




```
User
  |
API Gateway
  |
-----------------------------------------
| Movie Service | Train Service | Payment |
-----------------------------------------
      |               |              |
   DB-Movie       DB-Train       DB-Payment
```

Advantages

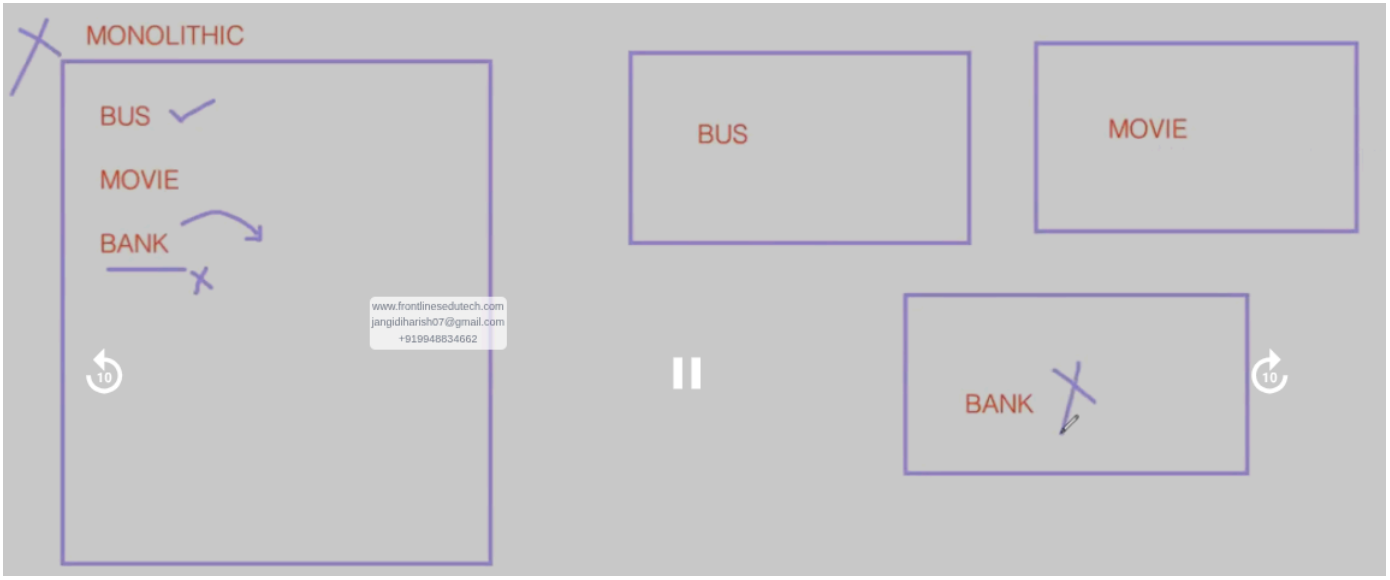
1. **Independent Deployment**
   Each service can be updated or restarted without affecting other services.
2. **Better Fault Isolation**
   If one service fails, the rest of the system can continue working.
3. **Independent Scaling**
   Only the services under heavy load need to be scaled.

Conclusion:

but coming to infra side maintain multiple servers for micro services difficult to manage as well as cost also ,

Managing many services across multiple servers can become complex.
To simplify deployment and management, technologies such as **Docker** are used. Docker allows each microservice to run inside its own **container**, ensuring consistent environments.
In larger systems, container orchestration tools like **Kubernetes** are used to manage multiple services automatically.

Both architectures have advantages depending on the size and complexity of the application.Monolithic architecture is easier to develop and manage in the early stages of a project. Microservices architecture becomes beneficial when applications grow large and require independent scaling, deployment, and fault isolation.



Why using Docker:

When building a modern application, it usually consists of three main components:

1. **Frontend** – The user interface (built using frameworks like **React** or **Angular**).
2. **Backend** – The server-side logic (for example using **Express.js** with **Node.js**).
3. **Database** – Used to store application data (such as **MongoDB**).

A typical application architecture looks like this:

Problem Without Docker

When developers build applications, the software must run on different environments such as:

- Developer laptops
- Testing servers
- Production servers

Each environment may have **different configurations**, such as:

- Different Node.js versions

- Different OS libraries
- Missing dependencies

Because of this, a common issue occurs:

> “It works on my machine, but it doesn't work on the server.”

For example:

- Developer machine → Node.js 18
- Server → Node.js 16

This mismatch can cause application failures.

Solution: Docker

Docker solves this problem by packaging the application with all its dependencies into a **container**.

A container includes:

- Application code
- Runtime (Node.js, Python, etc.)
- System libraries
- Dependencies

This ensures the application runs the **same way in every environment**.

Application with Docker

Instead of installing everything manually on servers, each component can run inside containers.

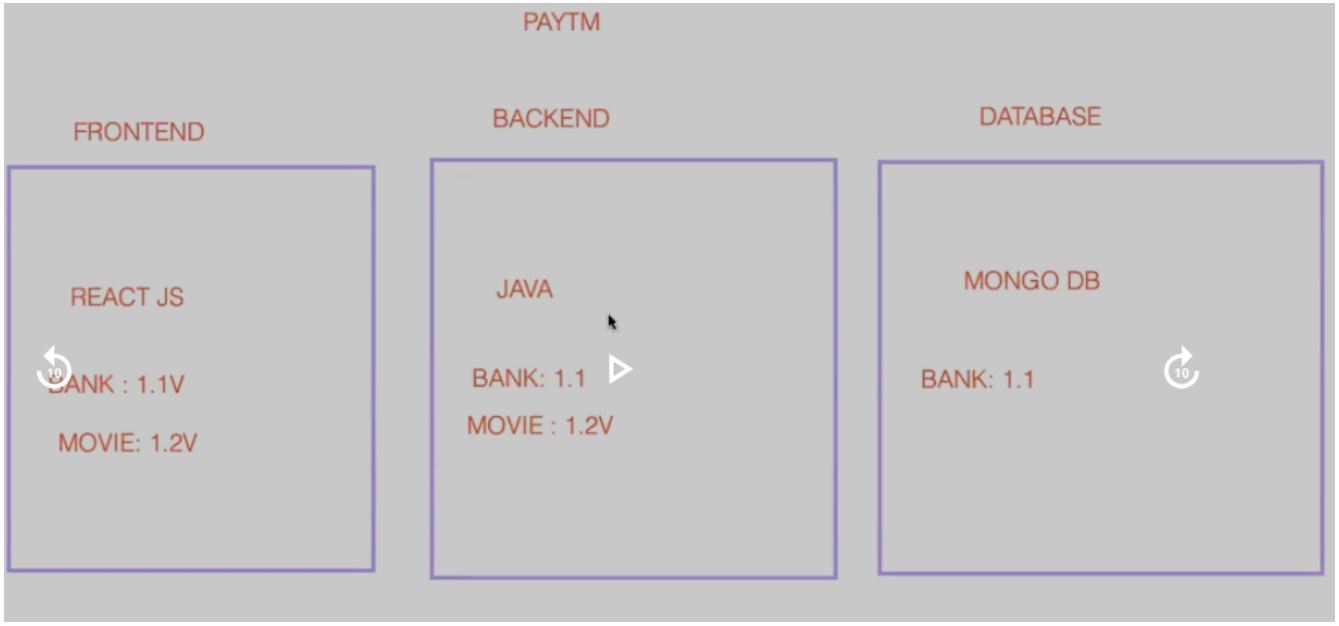
Example architecture:
Docker Container 1 → Frontend (React)
Docker Container 2 → Backend (Node.js + Express)
Docker Container 3 → Database (MongoDB)

Each container runs independently but communicates with others.

Benefits of Docker

1. **Environment Consistency**
   The application runs the same on developer machines, test servers, and production.

2. **Easy Deployment**
   Applications can be deployed quickly using containers.
3. **Isolation**
   Each service runs in its own environment without affecting others.
4. **Scalability**
   Containers can be easily replicated when traffic increases.



The **frontend** is the part of the application that users interact with in the browser.
In this example, the frontend is built using **React**.
Bank transaction module (Version 1.1) with react
Backend services include: Bank service (version 1.1)
.
Frontend → Backend API → Business Logic
The **database** stores application data such as:

- user information, transaction details,booking records

5. In this example, the database used is **MongoDB**.
6. The backend communicates with the database to:
   - read data
   - update records
   - store new transactions

Example flow:

Backend → MongoDB → Data Storage

Drawback: bank appliacation use react version1.1v version now iam using movie tickets in the it required react 1.2 version if different appliaion have different versions different dependcy
so here one iam using movie application of react 1.2 version it may effect some depency and

versions to bank for cleraly If the movie module requires React 1.2 but the bank module depends on React 1.1, installing the newer version may break the bank module. These problems may also occur when deploying the application on:Different developer laptops,Testing servers,Production servers Each environment may have different configurations and installed libraries. To overcome dependency and version conflicts, we use **Docker**.Docker packages the application along with all its required dependencies, libraries, and runtime versions into a container.This ensures the application runs consistently across different environments such as developer machines, testing servers, and production servers.

A container packages everything required to run the application, including:

- Application code
- Runtime environment
- Libraries
- Dependencies
- Specific software versions

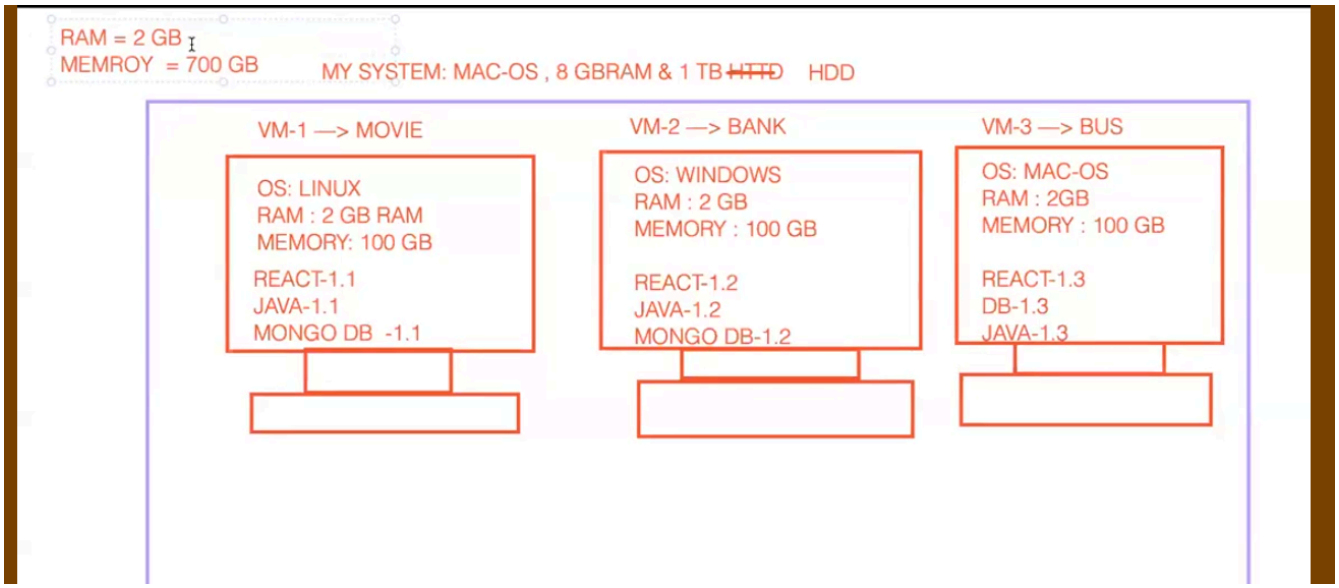
Because of this, the application runs **exactly the same in every environment**.

This eliminates problems related to:

- Version mismatches
- Dependency conflicts
- Environment differences

To solve this problem, the industry initially started using **Virtual Machines (VMs)**.
one system creating multiple virtual meachines

in one system we use mac-os , 8gbram & 1tb http.

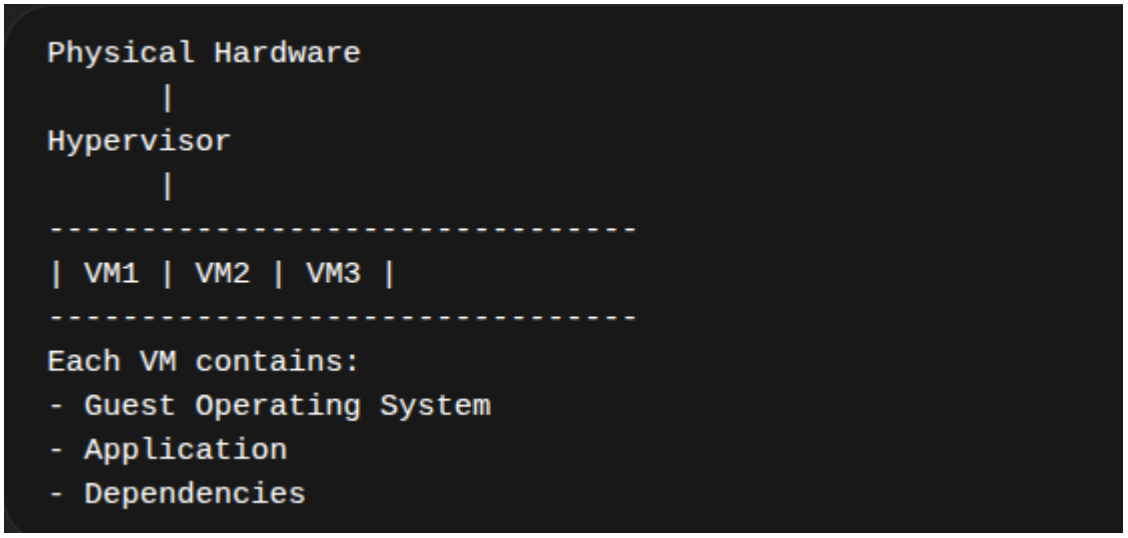


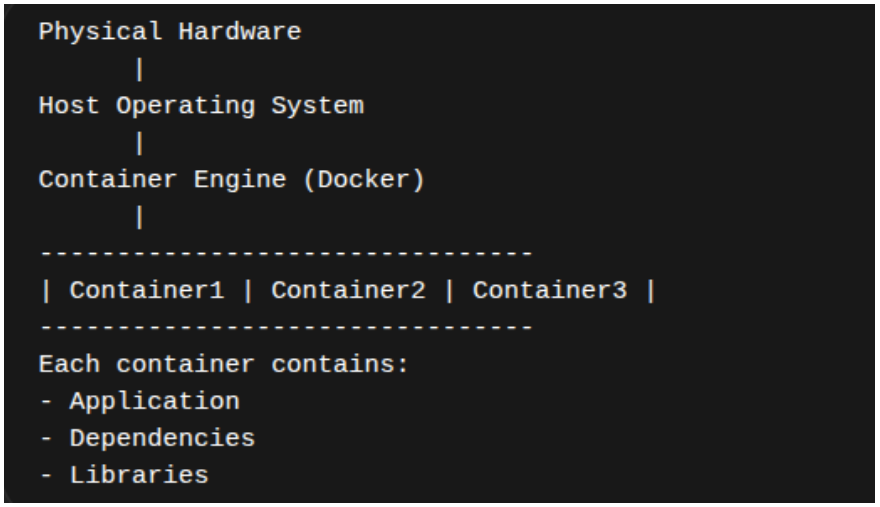
A **Virtual Machine** allows us to create multiple isolated operating systems on a single physical server.

1. This solves the **dependency conflict problem**, because each application runs in its own isolated environment.Although Virtual Machines solve dependency issues, they introduce several new challenges.,High Resource Consumption,Performance Issues,Complex Management,Higher Infrastructure Cost

so 8gb ram and 1tp HDD assign this to movie and abnk and bus only left2gb ram if i take anything it may not sufficient
performence also low and manging multiple vm is a complext as compared to ealry here versions issues resolved but coming to mange multiple vm is complex and cost effenciancy also increase and if server spikes it may be down if how mnay increase vm performence will slow and **To overcome the limitations of Virtual Machines, modern systems use containers**.

In a Virtual Machine environment, each VM runs its **own complete operating system**.
Virtualization platforms such as **VMware** or **Oracle VirtualBox** create separate virtual systems on top of the physical hardware.
Containers are different from Virtual Machines.Containers do **not include a full operating system**. Instead, they share the **host system's operating system kernel**.





```
Real Computer (Physical Hardware)
        |
Hypervisor (Manager Software)
        |
-------------------------------
| Small Computer | Small Computer |
|      VM1       |      VM2       |
-------------------------------
```

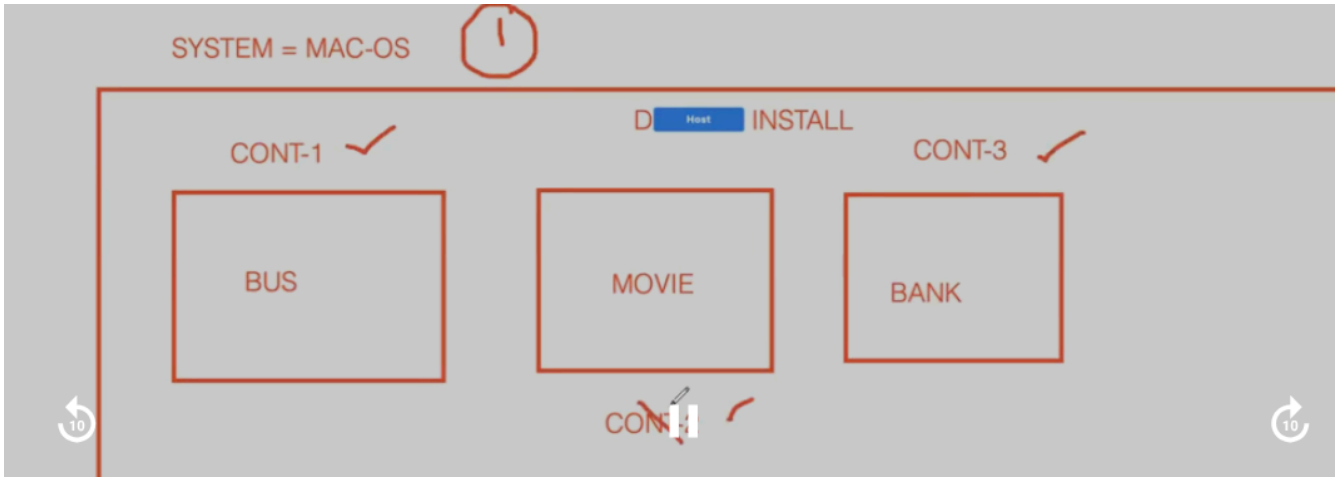
The **Hypervisor** takes the real computer and splits it into **multiple virtual computers (VMs)**.

Containers Explained (ELI5)

Imagine you have **one big kitchen**.

```
Kitchen (Computer)
      |
Chef (Operating System)
      |
Kitchen Manager (Docker)
      |
-------------------------------
| Lunchbox | Lunchbox | Lunchbox |
-------------------------------
```

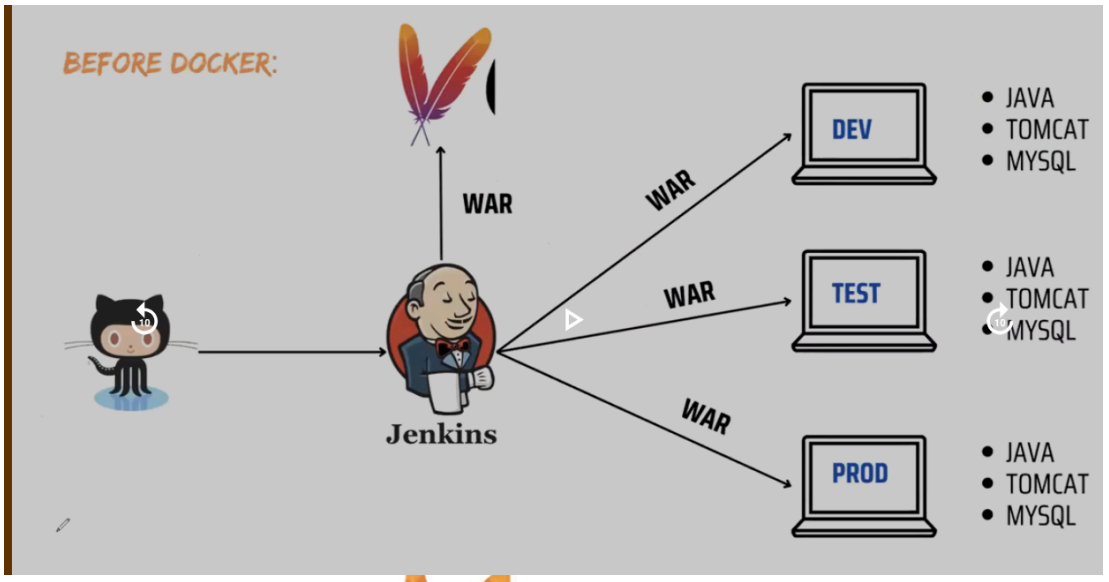
👉 Hypervisor creates full computers (VMs)
👉 Docker creates lightweight containers

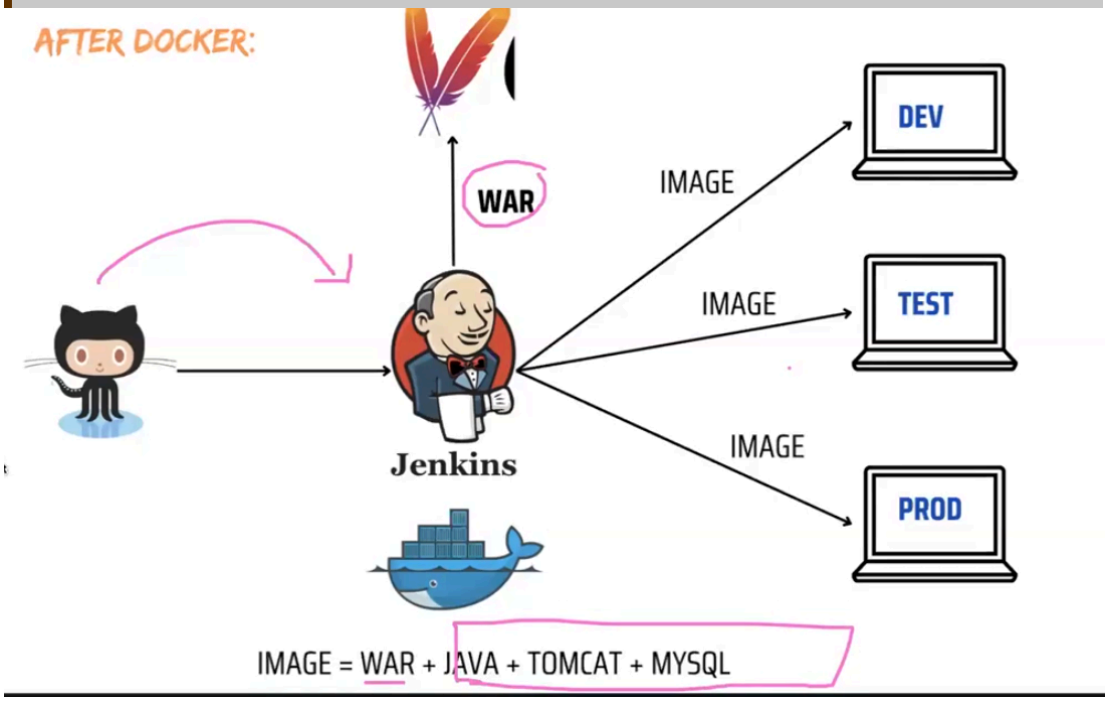


three containers i deploy one server through isolated we use
through the images will run containers

Lets learn about docker:

image= war+java+tomcat+mysql





in dev server i dont need to install all java,tomcat,mysql only i run image.

Docker

Docker is an open-source platform used to **build, deploy, and run applications using containers**. Containers allow applications to run in isolated environments while sharing the host operating system.

Key Characteristics of Docker

1. Open Source Platform

Docker is an open-source platform designed to **create, deploy, and manage containerized applications** efficiently.

2. Written in Go

Docker is developed using the **Go** programming language.

3. Uses Containers Instead of Virtual Machines

Docker runs applications inside **containers** on top of the host operating system.

Containers share the **Linux kernel** of the host system instead of creating a full operating system like Virtual Machines.

This makes containers:

- lightweight
- faster to start
- more resource efficient

4. Cross-Platform Installation

Docker can be installed on multiple operating systems such as:

- Linux
- Windows
- macOS

However, the **Docker Engine runs natively on Linux**, because containerization relies on Linux kernel features.

5. OS-Level Virtualization

Docker performs **Operating System level virtualization**, also known as **containerization**.

This is different from Virtual Machines, which use **hardware-level virtualization**.

6. Solves Environment Problems

Before Docker, developers often faced a common issue:

> "The application works on the developer's machine but fails on another system."

Docker solves this by packaging:

- application code
- dependencies
- libraries
- runtime environment

inside a container.

This ensures the application runs **consistently across different environments**.

7. Initial Release

Docker was first released in **March 2013** and was developed by **Solomon Hykes** and **Sebastien Pahl**.

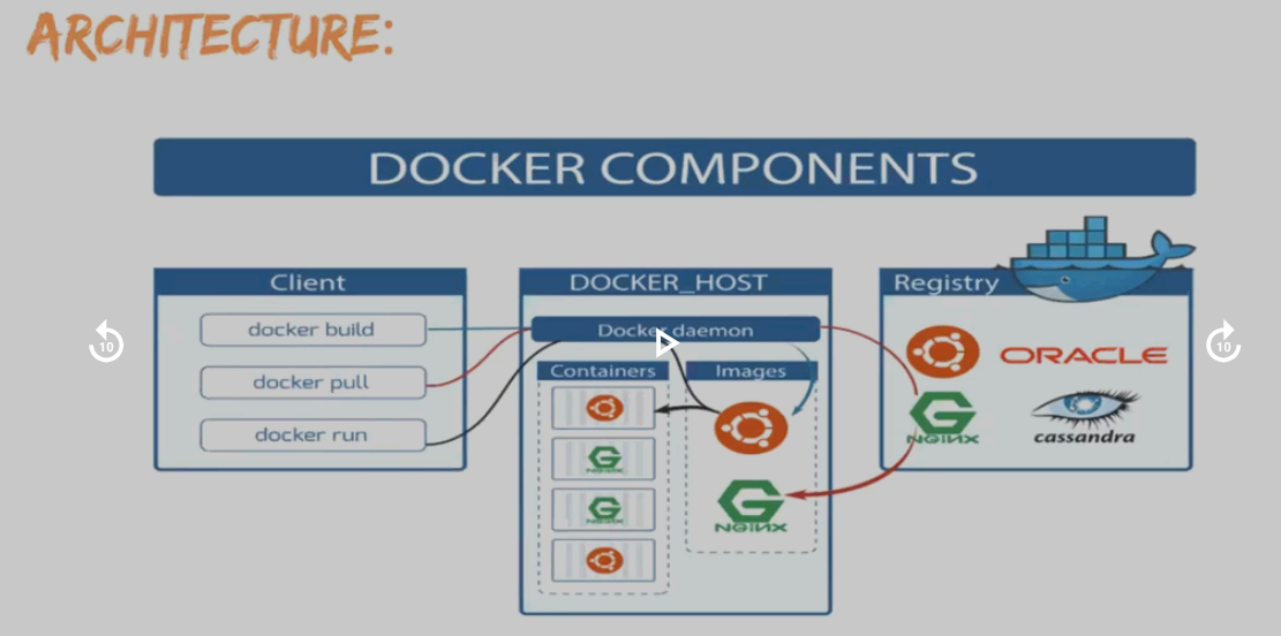
8. Container File System

Containers include necessary operating system libraries and files, but they are **much smaller than a full operating system** because they reuse the host system kernel.

One-Line Definition

Docker is a containerization platform that packages applications and their dependencies into lightweight containers to ensure consistent execution across different environments.

Docker Architecture



Docker client: It is the place where we perform the commands or The **Docker Client** is the interface through which users interact with Docker.

When you run commands like:

docker build

docker pull

docker run

these commands are sent to the **Docker Daemon** for execution.

Example:User → Docker Client → Docker Daemon

Docker Host: system where you install the docker in that we have daemon means it will run background

Daemon: daemon means it will run background

daemon has 4 components and It is responsible for

containers

images

volumes

network

The daemon receives instructions from the Docker client and performs the requested actions. In **Docker**, the client sends commands to the daemon, which builds images, runs containers, and interacts with registries like **Docker Hub**.

Registry: it is like how we we use git or github like that to store code we use git and store & share we use github where as same Docker we use images and docker hub store and share

Simple Docker Architecture Diagram